Entregable 4 — Integración y Despliegue Continuo

Catálogo de Videojuegos · Pipeline CI/CD con Jenkins

Sosa · Lapetina — Programación Avanzada, Universidad de Montevideo

Noviembre de 2026

1. Introducción

Este entregable implementa el ciclo de vida completo de una aplicación web sencilla, poniendo el foco —tal como pide la letra— **no en la funcionalidad ni en la apariencia de la aplicación, sino en el proceso de CI/CD**.

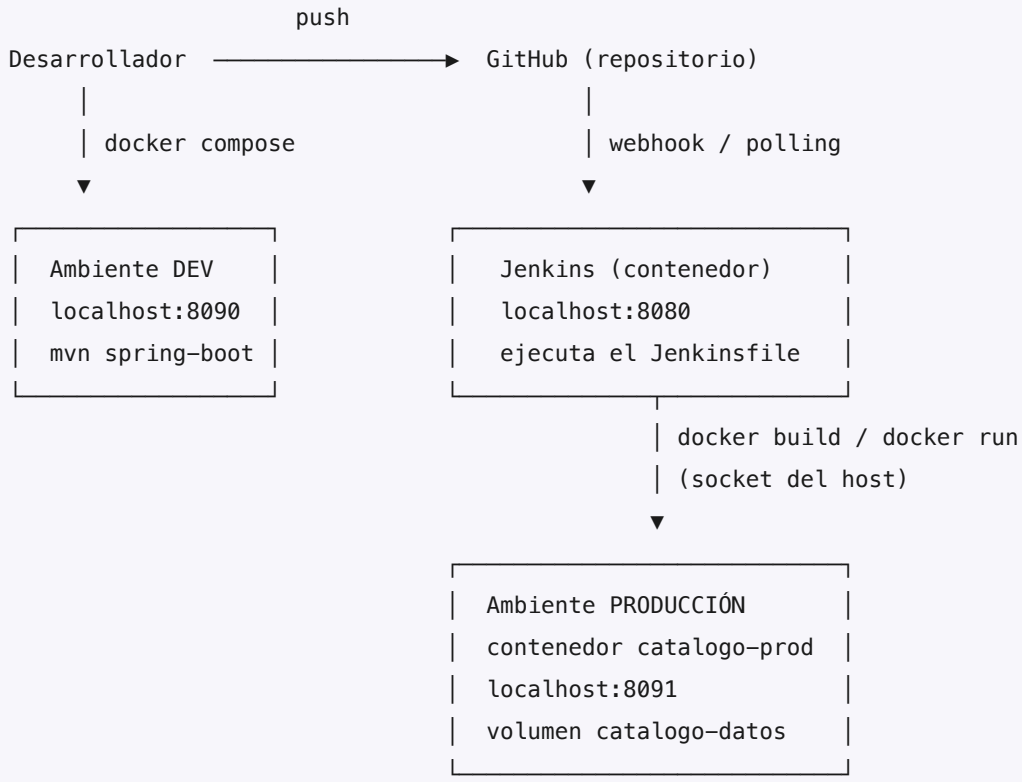
La aplicación elegida es un **catálogo de videojuegos** compuesto por dos módulos independientes. Sobre ella se construyó un pipeline declarativo de Jenkins, versionado junto al código (*pipeline as code*), que ante cada cambio en el repositorio de GitHub compila el sistema, corre las pruebas unitarias de cada módulo, corre las pruebas de integración, empaqueta la aplicación, construye una imagen de Docker y la despliega en un ambiente de producción simulado, verificando al final que la versión desplegada responda correctamente.

Repositorio: el código, el `Jenkinsfile`, los `Dockerfile` y este informe viven en el mismo repositorio de GitHub.

2. Arquitectura de la solución

2.1 Visión general

La solución tiene tres piezas que corren en contenedores separados sobre el mismo host:



Jenkins y la aplicación comparten la red de Docker `catalogo-red`, lo que le permite al pipeline verificar la salud del contenedor recién desplegado llamándolo por su nombre (`http://catalogo-prod:8090/actuator/health`) sin depender de la configuración de red del host.

2.2 Arquitectura de la aplicación

El sistema es un proyecto **Maven multi-módulo**. Cada módulo es un artefacto con su propio `pom.xml`, su propio código y su propia suite de pruebas unitarias:

```
catalogo-videojuegos (pom padre)
│
├── modulo-catalogo ............ MÓDULO 1
│     Juego                      entidad del dominio
│     RepositorioJuegos          contrato de persistencia (interfaz)
│     RepositorioJuegosArchivo   implementación sobre archivo JSON
│     ServicioCatalogo           reglas de negocio: alta, baja, consulta
│
├── modulo-clasificacion ....... MÓDULO 2   (depende del módulo 1)
│     Genero                     enum de géneros admitidos
│     ServicioClasificacion      normalización, filtrado, estadísticas
│
└── app-web ..................... APLICACIÓN (depende de los módulos 1 y 2)
      AplicacionCatalogo         arranque de Spring Boot
      ServicioAplicacion         integración entre ambos módulos
      CatalogoController         interfaz web (Thymeleaf)
      ApiCatalogoController      API REST
      ManejadorErroresApi        traducción de excepciones a HTTP 400
```

Las dependencias van en una sola dirección (`app-web → modulo-clasificacion → modulo-catalogo`), lo que permite probar cada módulo por separado y hace visible, en el reactor de Maven, el orden de construcción.

Punto de integración entre los módulos. El alta de un juego atraviesa los dos: el género que escribe el usuario se valida y normaliza con `ServicioClasificacion` (`"rpg"` → `"RPG"`, `"ACCIÓN"` → `"Accion"`) y recién entonces `ServicioCatalogo` aplica sus propias reglas (título obligatorio y no duplicado, año entre 1958 y el año actual + 5) y persiste. Las pruebas de integración ejercitan justamente ese camino.

2.3 Persistencia

La letra permite resolver la persistencia con una base de datos o con un archivo de texto. Se eligió un **archivo JSON** (`datos/juegos.json`) escrito con Jackson. En el contenedor de producción ese directorio es un **volumen de Docker** (`catalogo-datos-prod`), de modo que el catálogo sobrevive a cada despliegue: el pipeline destruye y recrea el contenedor, pero los datos quedan en el volumen.

2.4 Ambientes

Ambiente	Cómo se levanta	URL	Datos
Desarrollo	`./scripts/correr-dev.sh`	`localhost:8090`	`./datos/juegos.json`
Pruebas	Maven, durante las pruebas de integración	puerto aleatorio	`target/it-data/`
Producción simulada	desplegado por el pipeline de Jenkins	`localhost:8091`	volumen `catalogo-datos-prod`

La aplicación muestra en la cabecera el ambiente activo y la versión, para que durante la defensa quede claro qué instancia se está viendo.

3. El pipeline

3.1 Etapas

El `Jenkinsfile` es declarativo y está versionado en la raíz del repositorio:

#	Etapa	Qué hace	Si falla
1	Checkout	Clona el repo y registra commit, autor y mensaje	No hay build
2	Compilar	`./mvnw -DskipTests clean install` (install, no package: ver 6.2)	Producción intacta
3	Pruebas unitarias	`./mvnw surefire:test` (`*Test.java`), publica resultados JUnit	Producción intacta
4	Pruebas de integración	`./mvnw failsafe:integration-test failsafe:verify` (`*IT.java`)	Producción intacta
5	Archivar artefacto	Guarda `app-web.jar` con fingerprint	—
6	Imagen Docker	`docker build -t catalogo-videojuegos:${BUILD_NUMBER}`	Producción intacta
7	Desplegar	Reemplaza el contenedor `catalogo-prod` por la nueva imagen	Rollback
8	Smoke test	Verifica `/actuator/health` y `/api/juegos`	Rollback

La separación entre las etapas 3 y 4 se logra con la convención de nombres de Maven: **surefire** ejecuta los `*Test.java` (unitarios, sin contexto de Spring, en milisegundos) y **failsafe** ejecuta los `*IT.java` (integración, levantan la aplicación completa en un puerto real). Así el tablero de Jenkins muestra por separado qué tipo de prueba falló.

3.2 Disparadores

```
triggers {
    githubPush()                // webhook de GitHub
    pollSCM('H/2 * * * *')      // red de seguridad cada 2 minutos
}
```

Jenkins corre en `localhost`, por lo que GitHub no puede alcanzarlo desde internet para entregar el webhook. Se dejaron configurados los dos mecanismos: el webhook (que funciona si se expone Jenkins con un túnel, por ejemplo `ngrok http 8080`) y el *polling*, que consulta el repositorio cada dos minutos y arranca el build si detectó cambios. En la defensa se usa el polling, que no depende de la red de la facultad.

3.3 Estrategia ante fallos

- **Falla una etapa anterior al despliegue** (compilación o pruebas): el pipeline se detiene, el contenedor de producción nunca se toca y sigue sirviendo la versión anterior. Jenkins marca el build en rojo e indica el autor del commit culpable.
- **Falla el smoke test posterior al despliegue**: el bloque `post { failure }` detecta que ya había una imagen anterior desplegada y hace **rollback**, volviendo a levantar el contenedor con la imagen previa.

Cada imagen se etiqueta con el número de build (`catalogo-videojuegos:17`), lo que hace posible volver a cualquier versión anterior con un solo `docker run`.

3.4 Jenkins como código

El servidor Jenkins también está definido en el repositorio:

- `jenkins/Dockerfile`: parte de `jenkins/jenkins:lts-jdk21` y agrega el cliente de Docker, para que el pipeline pueda construir imágenes y levantar contenedores usando el daemon del host (patrón *Docker outside of Docker*, montando `/var/run/docker.sock`).
- `jenkins/plugins.txt`: los plugins que el pipeline necesita.
- `jenkins/casc.yaml`: *Configuration as Code*. Crea el usuario administrador y el job `catalogo-videojuegos` apuntando al repositorio, con sus disparadores. Esto evita tener que configurar Jenkins a mano y hace que el entorno sea reproducible en cualquier máquina del grupo.

Maven no se instala en la imagen: el proyecto incluye el **Maven Wrapper** (`./mvnw`), de modo que el pipeline usa exactamente la misma versión de Maven (3.9.9) que se usa en desarrollo.

4. Decisiones tomadas y justificación

4.1 Aplicación: catálogo de videojuegos

Se eligió el catálogo de videojuegos entre las opciones propuestas porque es el dominio con el modelo más pequeño (un juego es título, género, año e imagen) y porque la clasificación por género da naturalmente un **segundo módulo con entidad propia**, en lugar de ser una función suelta. Como la letra aclara que la funcionalidad no se evalúa, se priorizó un dominio chico que dejara el esfuerzo del lado del pipeline y que redujera la cantidad de cosas que pueden fallar durante la demostración en vivo.

4.2 Lenguaje y framework: Java 21 + Spring Boot 3.3

Impuesto por la letra (Java con Spring). Dentro de eso se eligió **Spring Boot** por tres razones concretas para un trabajo de CI/CD:

- produce un **jar ejecutable autocontenido**, que es exactamente lo que el pipeline necesita para armar una imagen de Docker mínima;
- trae **Spring Boot Actuator**, cuyo endpoint `/actuator/health` se usa como *smoke test* del despliegue y como `HEALTHCHECK` del contenedor;
- su soporte de pruebas (`@SpringBootTest` con puerto aleatorio) permite escribir pruebas de integración reales sin infraestructura adicional.

Se compila con `--release 21` (LTS) aunque el JDK instalado sea más nuevo, para que el mismo bytecode corra en la imagen `eclipse-temurin:21-jre` que usa producción.

4.3 Maven multi-módulo

La letra pide "un sistema formado por varios módulos". Se usó un proyecto Maven multi-módulo en lugar de un único proyecto con paquetes separados porque:

- obliga a que las dependencias entre módulos sean explícitas y unidireccionales;
- cada módulo tiene su propia suite de pruebas unitarias, y el reactor las ejecuta por separado: en el log del pipeline se ve qué módulo falló;
- es la estructura que después permite publicar los módulos como artefactos independientes.

Se agregó el **Maven Wrapper** al repositorio para que ni Jenkins ni los integrantes del grupo tengan que instalar Maven, y para que todos usen la misma versión.

4.4 Pruebas: surefire para unitarias, failsafe para integración

En lugar de distinguir los dos tipos de prueba con etiquetas, se usó la convención estándar de Maven: `*Test.java` lo ejecuta surefire y `*IT.java` lo ejecuta failsafe. Es una decisión barata que da dos beneficios: el pipeline puede correr cada conjunto en una etapa distinta sin recompilar, y Jenkins publica dos reportes JUnit separados.

Las pruebas unitarias no levantan Spring: usan un repositorio en memoria escrito a mano como doble de prueba. Las de integración sí levantan la aplicación completa sobre un puerto real y la consultan por HTTP, con un archivo de datos temporal (`target/it-data/`) para no ensuciar los datos de desarrollo.

4.5 Persistencia en archivo JSON

Se descartó una base de datos porque habría agregado un contenedor más, credenciales y migraciones al ambiente de producción simulado, sin aportar nada al objetivo del entregable. El archivo JSON montado como volumen de Docker demuestra el punto importante —que los datos sobreviven a los despliegues— con una fracción de la complejidad.

4.6 Ambiente de producción simulado en Docker

La letra admite una carpeta local, un contenedor u otra alternativa. Se eligió Docker porque:

- el despliegue es **reproducible**: la misma imagen corre igual en cualquier máquina del grupo y en la de la defensa;
- permite versionar cada despliegue (`catalogo-videojuegos:${BUILD_NUMBER}`) y por lo tanto hacer *rollback* real;
- deja el ambiente de producción visiblemente separado del de desarrollo (otro puerto, otros datos, otra etiqueta en la cabecera de la página).

4.7 Jenkins en contenedor, configurado como código

Jenkins corre en un contenedor construido desde el repositorio. Para que el pipeline pueda a su vez manejar contenedores se montó el socket del daemon del host (*Docker outside of Docker*) en vez de usar *Docker in Docker*, que es más frágil y pesado. La configuración del servidor (usuario administrador y definición del job) está en `jenkins/casc.yaml` con el plugin *Configuration as Code*, de manera que levantar el entorno completo en otra máquina sea un único comando y no una secuencia de clics.

5. Instructivo para ejecución

5.1 Requisitos

- **Docker Desktop** en ejecución (único requisito obligatorio).
- **JDK 21 o superior**, solo si se quiere compilar fuera de Docker. Maven no es necesario: el repositorio incluye `./mvnw`.

5.2 Clonar el repositorio

```
git clone https://github.com/USUARIO/catalogo-videojuegos.git
cd catalogo-videojuegos
```

5.3 Correr la aplicación en desarrollo

```
./scripts/correr-dev.sh
```

La aplicación queda en http://localhost:8090 con la etiqueta `DESARROLLO`.

El script encadena dos comandos, y los dos son necesarios: `./mvnw -DskipTests install`, que deja los jars de los módulos en el repositorio local de Maven, y después `./mvnw -pl app-web spring-boot:run`. Un único `./mvnw -pl app-web -am spring-boot:run` falla con *Unable to find a suitable main class*, porque `-am` arrastra al POM padre al reactor y el plugin intenta ejecutarlo también sobre él.

5.4 Correr las pruebas

```
./scripts/probar-todo.sh      # equivale a ./mvnw clean verify
```

Ejecuta las pruebas unitarias de los tres módulos y después las de integración.

5.5 Levantar Jenkins

```
export REPO_URL=https://github.com/USUARIO/catalogo-videojuegos.git
./scripts/levantar-jenkins.sh
```

El script crea la red `catalogo-red` y levanta Jenkins en http://localhost:8080 (usuario `admin`, contraseña `admin`). El job `catalogo-videojuegos` ya queda creado apuntando al repositorio indicado, con el *polling* activo.

5.6 Ejecutar el pipeline

Alcanza con hacer *push* al repositorio: Jenkins lo detecta en menos de dos minutos y arranca el build. También se puede forzar con **Build Now** desde la interfaz.

Al terminar, la nueva versión queda desplegada en http://localhost:8091 con la etiqueta `PRODUCCIÓN`.

5.7 Comandos auxiliares

```
./scripts/estado.sh            # estado de los contenedores y salud de producción
./scripts/desplegar-local.sh   # desplegar a producción sin pasar por Jenkins
./scripts/detener-todo.sh      # detener Jenkins y producción
```

6. Errores encontrados al ejecutar el pipeline y cómo se resolvieron

Esta sección recoge los problemas reales que aparecieron al poner el pipeline en funcionamiento, en el orden en que los fuimos encontrando.

6.1 Maven no estaba instalado y compilarlo tardaba demasiado

Síntoma. El proyecto no compilaba en la máquina de desarrollo: no había `mvn`, y al instalarlo con Homebrew el gestor empezó a compilar dependencias desde el código fuente (cmake, entre otras) porque no había paquetes precompilados para esa versión de macOS.

Causa. Depender de una herramienta instalada manualmente en cada máquina.

Solución. Se agregó el **Maven Wrapper** al repositorio (`mvnw`, `mvnw.cmd`, `.mvn/wrapper/`). El wrapper descarga por sí solo la versión de Maven que declara el proyecto (3.9.9). Esto resolvió el problema de entorno y, de paso, eliminó un requisito de la imagen de Jenkins: el pipeline usa `./mvnw`, no un Maven instalado en el servidor, así que desarrolladores y CI construyen con exactamente la misma versión.

6.2 El pipeline fallaba al resolver las dependencias entre módulos

Síntoma. El build pasaba la etapa *Compilar* y fallaba en *Pruebas unitarias*:

```
[ERROR] Failed to execute goal on project modulo-clasificacion:
        Could not resolve dependencies for project uy.edu.um.pa:modulo-clasificacion:jar:1.0
[ERROR] dependency: uy.edu.um.pa:modulo-catalogo:jar:1.0.0 (compile)
[ERROR]   Could not find artifact uy.edu.um.pa:modulo-catalogo:jar:1.0.0 in central
```

Lo llamativo es que el mismo proyecto compilaba y pasaba todas las pruebas localmente con `mvn clean verify`.

Causa. Este fue el error más instructivo del entregable. El pipeline separa las pruebas en dos etapas invocando los plugins directamente (`mvnw surefire:test` y `mvnw failsafe:integration-test`). Cuando se invoca un *goal* suelto, Maven no ejecuta el ciclo de vida completo, y por lo tanto los módulos del reactor no tienen un artefacto empaquetado que ofrecerse entre sí: `modulo-clasificacion` sale a buscar el jar de `modulo-catalogo` al repositorio local, no lo encuentra, y lo busca en Maven Central, donde obviamente no existe. Localmente nunca se notó porque `mvn clean verify` corre el ciclo completo y la resolución se hace dentro del reactor.

Solución. Cambiar la etapa *Compilar* de `clean package` a **`clean install`**, de modo que los jars de los dos módulos queden instalados en el repositorio local de Maven antes de que las etapas siguientes los necesiten. El repositorio local vive en el volumen persistente de Jenkins (`-Dmaven.repo.local=/var/jenkins_home/.m2/repository`), así que además sirve de caché entre builds.

6.3 Jenkins se negaba a clonar un repositorio local

Síntoma. Mientras probábamos el pipeline antes de tener el repositorio en GitHub, apuntamos el job a un repositorio local dentro del contenedor y el checkout abortó:

```
ERROR: Checkout of Git remote 'file:///var/jenkins_home/repo-demo.git' aborted
       because it references a local directory, which may be insecure.
```

Causa. El plugin de Git de Jenkins bloquea por seguridad los checkouts desde rutas del sistema de archivos (un `Jenkinsfile` malicioso podría leer directorios del servidor). Se habilitan con la propiedad de sistema `hudson.plugins.git.GitSCM.ALLOW_LOCAL_CHECKOUT`.

Solución. En lugar de bajar una protección del servidor, servimos el repositorio de prueba por protocolo Git (`git daemon`) y apuntamos el job a `git://...`. Con el repositorio real de GitHub el problema no se presenta, porque la URL es remota.

6.4 Git rechazaba el repositorio por “dubious ownership”

Síntoma. Al clonar dentro del contenedor:

```
fatal: detected dubious ownership in repository at '/tmp/proyecto/.git'
```

Causa. El contenedor de Jenkins corre como `root` y el repositorio copiado pertenecía a otro usuario; desde la versión 2.35.2, Git se niega a operar sobre repositorios de otro dueño.

Solución. `git config --global --add safe.directory "*"` dentro del contenedor. Es el mismo mensaje que aparece con frecuencia al montar carpetas del host en contenedores, así que valía la pena dejarlo documentado.

6.5 El primer build contra GitHub clonó la URL anterior

Síntoma. Después de crear el repositorio en GitHub y relevantar Jenkins con la variable `REPO_URL` apuntando al repositorio real, el primer build falló en el checkout intentando clonar la URL de ejemplo que se había usado antes:

```
> git fetch --tags --force --progress -- https://github.com/EJEMPLO/catalogo-videojuegos.git
stderr: remote: Invalid username or token.
fatal: Authentication failed
```

Lo desconcertante es que la configuración del job ya mostraba la URL correcta.

Causa. Jenkins persiste la cola de builds (`queue.xml`) entre reinicios. El trigger de *polling* había encolado un build con la configuración vieja antes de que apagáramos el contenedor, y al volver a arrancar ese build se ejecutó primero, con la URL anterior. La actualización que hace *Configuration as Code* al arrancar no alcanza a los builds que ya estaban en la cola.

Solución. Descartar ese primer build y volver a disparar: el siguiente ya clonó del repositorio correcto y terminó en verde. Conviene tenerlo presente el día de la defensa: si se cambia la configuración del job y hay un build encolado, el primer resultado puede corresponder a la configuración anterior.

7. Demostración: los dos casos de la defensa

7.1 Caso 1 — Integración exitosa

1. Se modifica algo visible del código, por ejemplo agregar un género nuevo al enum `Genero` o cambiar el título de la página.
2. `git commit` y `git push` a `main`.
3. En menos de dos minutos Jenkins detecta el cambio (o se dispara el webhook) y arranca el build. El tablero muestra las ocho etapas pasando en verde.
4. Al terminar la etapa *Desplegar a producción*, el contenedor `catalogo-prod` se recrea con la imagen nueva y el *smoke test* confirma que responde.
5. Se recarga http://localhost:8091 y se ve el cambio en producción, con el número de versión actualizado en la cabecera. Los juegos cargados siguen estando: el volumen de datos sobrevivió al despliegue.

7.2 Caso 2 — Fallo en las pruebas

1. Se introduce un error a propósito. Por ejemplo, en `ServicioClasificacion.normalizar` se devuelve el texto sin normalizar, lo que rompe la prueba unitaria `normalizaGenero` y la prueba de integración `filtradoPorGenero`.
2. Se hace *push*. Jenkins arranca el build.

3. Las etapas *Checkout* y *Compilar* pasan; la etapa **Pruebas unitarias** falla. El pipeline se detiene ahí: **no se construye la imagen ni se despliega nada**.
4. Jenkins muestra en rojo la etapa que falló, publica el reporte JUnit con el nombre exacto de la prueba (`ServicioClasificacionTest.normalizaGenero`, esperaba `"RPG"` y recibió `"rpg"`) e informa el autor y el mensaje del commit responsable.
5. **Producción no se vio afectada**: http://localhost:8091 sigue sirviendo la versión anterior, que es el comportamiento que se busca.
6. Procedimiento de resolución: el desarrollador corrige el código (o revierte el commit con `git revert`), vuelve a hacer *push*, y el pipeline corre de nuevo. Recién cuando todas las pruebas pasan, la nueva versión llega a producción.

Este segundo caso es el que justifica tener el pipeline: el error quedó contenido entre el *push* y el despliegue, sin intervención manual y sin que nadie tuviera que acordarse de correr las pruebas.

8. Conclusiones

El trabajo dejó una separación muy clara entre dos cosas que en los entregables anteriores venían mezcladas: **escribir código que funciona** y **tener un proceso que garantice que lo que llega al ambiente de producción funciona**. La aplicación en sí es deliberadamente modesta; el valor está en que ningún cambio llega a producción sin haber pasado por compilación, pruebas unitarias de cada módulo, pruebas de integración de todo el sistema y una verificación de salud posterior al despliegue, todo sin intervención manual.

Algunas conclusiones concretas:

- **El pipeline como código vale la pena.** Tener el `Jenkinsfile` versionado junto al proyecto significa que el proceso de construcción evoluciona con el código y que cualquier cambio al pipeline queda registrado en el historial de Git, revisable como cualquier otro cambio. Llevar la misma idea a la configuración del servidor (`jenkins/casc.yaml`) hizo que levantar el entorno completo en otra máquina fuera un solo comando.
- **Separar pruebas unitarias de pruebas de integración cambia el diagnóstico.** Cuando falla una etapa, el tablero dice inmediatamente si se rompió la lógica de un módulo o la interacción entre módulos. Esa distinción no cuesta nada (es una convención de nombres) y ahorra tiempo cada vez que algo falla.
- **El ambiente de producción simulado en Docker obligó a pensar en el estado.** Como cada despliegue destruye y recrea el contenedor, los datos tenían que vivir fuera de él; separarlos en un volumen fue lo que convirtió el despliegue en algo repetible sin perder el catálogo cargado.

8.1 Dificultades encontradas

- **Docker dentro de Docker.** Entender que el contenedor de Jenkins no debía *contener* un daemon de Docker sino *usar* el del host fue el concepto que más costó. Una vez claro, se resolvió montando `/var/run/docker.sock`, pero obliga a correr el contenedor de Jenkins como `root` en Docker Desktop para poder escribir en ese socket.
- **La red entre contenedores.** Verificar el despliegue desde Jenkins no es trivial: dentro del contenedor de Jenkins, `localhost` no es el host, así que el smoke test no puede apuntar a

`localhost:8091`. Se resolvió creando una red de Docker compartida (`catalogo-red`) a la que se conectan Jenkins y la aplicación, de modo que el smoke test llama al contenedor por su nombre (`http://catalogo-prod:8090`). Esto además hace que el pipeline funcione igual en Linux y en Docker Desktop.

- **El webhook de GitHub contra un Jenkins local.** GitHub no puede alcanzar `localhost:8080`. Se dejó el webhook configurado para cuando se expone Jenkins con un túnel, y el *polling* del SCM como mecanismo de respaldo para la demostración.
- **Tiempos del primer build.** La primera ejecución del pipeline es notablemente más lenta que las siguientes porque Maven baja todas las dependencias dentro del contenedor. Se mitigó dejando el repositorio local de Maven en el volumen persistente de Jenkins (`/var/jenkins_home/.m2/repository`), de modo que los builds siguientes reutilizan la caché.

8.2 Trabajo futuro

Lo primero que agregaríamos es un **ambiente intermedio de UAT** con una aprobación manual en Jenkins (`input`) antes de promover a producción, que es el camino natural de este pipeline. Después, publicar las imágenes en un registro en lugar de dejarlas en el daemon local, y reemplazar el `docker run` del despliegue por un `docker compose up` con el archivo versionado, para que la definición del ambiente de producción también sea código.

Anexo A — Evidencia de ejecución del pipeline

A.1 Build exitoso (caso 1)

Salida resumida de una ejecución completa del pipeline:

```
[Pipeline] { (Checkout)
Commit ba5b8a7 de Sosa Lapetina: Corrige el pipeline: instalar los modulos
[Pipeline] { (Compilar)
[INFO] BUILD SUCCESS
[Pipeline] { (Pruebas unitarias)
[INFO] Tests run: 9, Failures: 0 -- in uy.edu.um.pa.catalogo.ServicioCatalogoTest
[INFO] Tests run: 3, Failures: 0 -- in uy.edu.um.pa.catalogo.RepositorioJuegosArchivoTest
[INFO] Tests run: 6, Failures: 0 -- in uy.edu.um.pa.clasificacion.GeneroTest
[INFO] Tests run: 9, Failures: 0 -- in uy.edu.um.pa.clasificacion.ServicioClasificacionTest
[INFO] Tests run: 3, Failures: 0 -- in uy.edu.um.pa.web.ServicioAplicacionTest
[Pipeline] { (Pruebas de integracion)
[INFO] Tests run: 6, Failures: 0 -- in uy.edu.um.pa.web.CatalogoApiIT
[INFO] Tests run: 4, Failures: 0 -- in uy.edu.um.pa.web.PaginaWebIT
[Pipeline] { (Archivar artefacto)
[Pipeline] { (Construir imagen Docker)
[Pipeline] { (Desplegar a produccion)
Version actualmente desplegada: catalogo-videojuegos:2
+ docker rm -f catalogo-prod
+ docker run -d --name catalogo-prod --network catalogo-red -p 8091:8090 \
      -v catalogo-datos-prod:/app/datos -e APP_AMBIENTE=PRODUCCION \
      catalogo-videojuegos:3
[Pipeline] { (Smoke test de produccion)
Esperando a que la aplicacion responda...
La aplicacion esta arriba (intento 4)
La API responde correctamente
[Pipeline] { (Declarative: Post Actions)
 DESPLIEGUE EXITOSO  -  build #3  -  commit ba5b8a7
 Produccion: http://localhost:8091
Finished: SUCCESS
```

Totales: 30 pruebas unitarias y 10 pruebas de integración, en 8 etapas.

A.2 Build fallido (caso 2)

Se modificó `ServicioClasificacion.normalizar` para que devolviera el texto sin normalizar. Resultado real del pipeline:

```
[Pipeline] { (Checkout)
[Pipeline] { (Compilar)
[Pipeline] { (Pruebas unitarias)
[ERROR] Tests run: 9, Failures: 2 <<< FAILURE! -- in ServicioClasificacionTest
[ERROR] ServicioClasificacionTest.normalizaGenero -- <<< FAILURE!
        org.opentest4j.AssertionFailedError: expected: <RPG> but was: <rpg>
[ERROR] ServicioClasificacionTest.falloAlNormalizar -- <<< FAILURE!
[ERROR] Tests run: 15, Failures: 2, Errors: 0, Skipped: 0

[Pipeline] { (Pruebas de integracion)
Stage "Pruebas de integracion" skipped due to earlier failure(s)
[Pipeline] { (Archivar artefacto)
Stage "Archivar artefacto" skipped due to earlier failure(s)
[Pipeline] { (Construir imagen Docker)
Stage "Construir imagen Docker" skipped due to earlier failure(s)
[Pipeline] { (Desplegar a produccion)
Stage "Desplegar a produccion" skipped due to earlier failure(s)
[Pipeline] { (Smoke test de produccion)
Stage "Smoke test de produccion" skipped due to earlier failure(s)

[Pipeline] { (Declarative: Post Actions)
 PIPELINE FALLIDO  -  build #4  -  commit ba7a796
 Autor del cambio: Sosa Lapetina
No se modifico produccion: el fallo ocurrio antes del despliegue.
Finished: FAILURE
```

Verificación posterior sobre el ambiente de producción:

```
$ docker inspect -f '{{.Config.Image}}' catalogo-prod
catalogo-videojuegos:3            ← sigue la versión del build anterior

$ curl -s http://localhost:8091/actuator/health
{"status":"UP", ...}
```

Es decir: el commit defectuoso **nunca llegó a producción**, el pipeline informó qué prueba falló y con qué valor, y el servicio siguió disponible durante todo el episodio. Al revertir el cambio y volver a ejecutar, el pipeline terminó nuevamente en verde.